# Engineering Reliable LLM-Based Data Analysis: An Empirical Study of Schema Grounding, Planning, Verification, and SQL Repair

**Mevin Jose**

August 2026

**Abstract**

While Large Language Models (LLMs) demonstrate notable code-generation capabilities, translating natural language questions into reliable analytical SQL over relational databases (Text-to-SQL) remains brittle in practice. Queries frequently fail due to schema hallucinations, invalid join topologies, aggregation grain mismatch, and SQL dialect incompatibilities. In this paper, we present an empirical reliability study investigating which architectural mechanisms improve the reliability of LLM-generated analytical SQL, and what failure modes persist across grounding, planning, structural verification, and automated repair. We evaluate a multi-stage reliability pipeline on a frozen 500-query benchmark across 8 business domains over a public relational e-commerce data warehouse (the Olist dataset, 9 tables, 100,000+ orders). Our system achieves a **73.4% Result Equivalence Rate under the study comparator** (367/500 queries, 95% Wilson Score CI: [69.3%, 77.2%]), **31.0% Exact Match**, and **100.0% SQL Execution Success** with a mean latency of 64.04s ($p50$: 56.47s, $p95$: 121.92s). In a controlled 4-way 100-query ablation, activating AST-based structural verification significantly improves result equivalence over unverified planners (Config C 26.0% vs. Config B 15.0%, McNemar exact $p = 0.0192$, OR = 3.75). An exhaustive audit of all 101 repair events reveals that automated self-repair is a double-edged mechanism: while repair maintained 96.0% syntactic validity and preserved 49 already-correct queries, it yielded only 4 genuine recoveries while inducing 22 false-positive regressions. In a controlled 50-query synthetic perturbation robustness study across 5 vectors, the pipeline remains resilient to paraphrasing, synonyms, and ranking variants, but degrades under typographical noise (57.1% retention). We conclude that conservative, uncertainty-aware structural verification is preferable to unrestricted automated repair, and document the persistent structural failure modes across complex analytical queries.

## 1 Introduction and Research Questions

Natural language interfaces to relational databases (Text-to-SQL) have received widespread attention for democratizing analytical access [4,11]. However, translating ambiguous business questions into sound SQL over normalized multi-table schemas reveals a fundamental operational reality:

*Executable SQL is not necessarily reliable analytical SQL.*

A query may compile cleanly and execute without database runtime exceptions, yet return subtly corrupted figures due to several systemic failure modes:

1. **Schema Grounding Hallucinations:** LLMs frequently invent nonexistent columns (e.g., querying `order_amount` instead of `price`), guess unindexed foreign keys, or join tables across disconnected topologies.

2. **Join-Path & Topology Errors:** In multi-table relational schemas, joining distant entities without necessary intermediate bridging tables creates invalid query semantics or cartesian fan-out.

3. **Aggregation Grain Inconsistencies:** Computing metrics across mismatched dimensional grains (e.g., joining order items before aggregating customer attributes) produces inflated totals due to join fan-out multiplication.

4. **Filtering & Ranking Errors:** Queries often omit critical domain predicates (e.g., `order_status = 'delivered'`) or invert ordering directions in ranking clauses.

5. **Dialect & Runtime Failures:** Syntactic incompatibilities across SQL engines (e.g., SQLite's `strftime` vs. PostgreSQL's `DATE_TRUNC`) cause execution failures when models mix dialect idioms.

To evaluate how these failure modes can be mitigated, we study a multi-stage architecture that decomposes query synthesis into explicit schema grounding, structured query planning, deterministic plan validation, AST-based structural verification, and closed-loop repair. Rather than treating Text-to-SQL as end-to-end string generation, this study investigates the marginal contribution and empirical limitations of each stage.

We formulate the central research question:

> *Which architectural mechanisms improve the reliability of LLM-generated analytical SQL, and what failure modes remain after grounding, planning, verification, and repair?*

Specifically, we address five explicit research questions:

- **RQ1 (Overall Reliability):** Does the multi-stage reliability pipeline improve result equivalence on complex relational queries?
- **RQ2 (Marginal Verification Effect):** What is the marginal impact of deterministic AST structural verification over unverified query planning?
- **RQ3 (Automated Repair Dynamics):** Does automated repair genuinely recover incorrect queries without degrading already-correct ones?
- **RQ4 (Failure Taxonomy):** What structural failure modes dominate the remaining non-equivalent queries?
- **RQ5 (Controlled Perturbation Robustness):** How sensitive is the system to controlled lexical, temporal, and typographical perturbations?

## 2 Related Work

**Text-to-SQL Benchmarks and Decomposition Methods.** Benchmarks such as Spider [11] and BIRD [4] evaluate LLMs on multi-table joins, nested subqueries, and execution accuracy. Decomposition frameworks such as DIN-SQL [7] and MAC-SQL [9] show that dividing generation into sub-problems (schema linking, classification, SQL generation, correction) improves performance over monolithic prompting. Our work builds upon this paradigm by introducing deterministic, AST-level structural verification gates that inspect query structure prior to execution.

**Schema Linking and Retrieval-Augmented Generation.** Retrieval-Augmented Generation (RAG) [1,3] grounds LLM generation in external knowledge. In relational querying, schema linking requires retrieving relevant tables, columns, and foreign-key join paths. While prior approaches rely primarily on dense embeddings or BM25 keyword matching, we evaluate hybrid retrieval augmented with explicit foreign-key graph traversal to preserve schema connectivity.

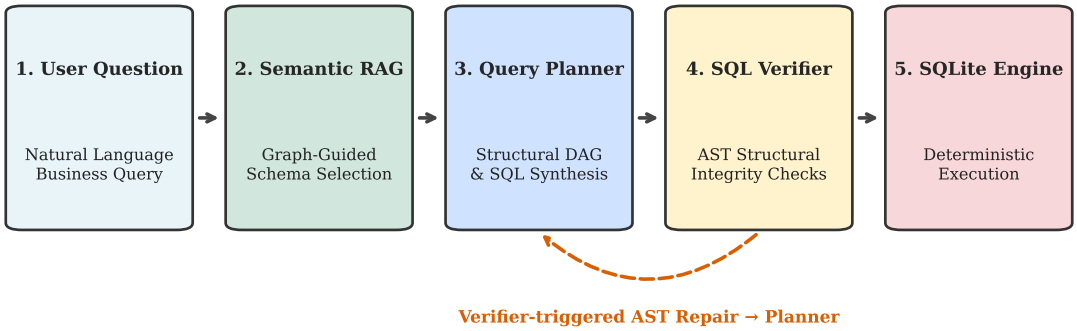


Figure 1: Reliability-Oriented Multi-Stage Architecture: Natural language questions are translated into verified execution results through graph-guided schema RAG, structured DAG planning, deterministic plan validation, AST structural verification, and verifier-triggered repair.

**Agentic Architectures and Self-Correction Hazards.** Iterative reasoning architectures such as ReAct [10] and Reflexion [8] leverage feedback loops for self-correction. In Text-to-SQL, execution-guided self-correction feeds compiler errors back to the LLM [2]. However, our empirical audit demonstrates that relying solely on execution feedback is hazardous: 96% of post-repair queries execute cleanly without engine exceptions, yet over 21% suffer from false-positive regressions that degrade previously correct queries.

# 3 Reliability-Oriented System Architecture

Our architecture decomposes analytical SQL synthesis into a 5-stage pipeline (Figure 1):

## 3.1 Graph-Guided Semantic Schema RAG

Rather than passing an entire database schema into the LLM context, our retriever combines BM25 keyword matching, dense embedding retrieval via SentenceTransformers, and foreign-key graph traversal. Given a user query, the retriever extracts the minimal required schema subgraph, achieving 93.1% macro precision and 95.3% macro recall across the 500-query benchmark.

## 3.2 Structured DAG Query Planning & Deterministic Plan Validation

Before emitting SQL code, the planner synthesizes a structural JSON DAG declaring target metrics, grain dimensions, required tables, and explicit join paths. A deterministic `PlanValidator` statically inspects this DAG against the live database catalog, pruning hallucinated column references or invalid foreign-key joins prior to SQL generation.

## 3.3 AST-Based SQL Structural Verification & Closed-Loop Repair

Generated SQL statements are parsed into Abstract Syntax Trees (AST) using SQLGlot [5]. The `SQLSemanticVerifier` statically enforces:

- **Dialect Compliance:** Translating dialect-specific functions (`DATEDIFF`, `DATE_TRUNC`) into canonical SQLite expressions (`strftime`, `julianday`).

- **Join Fan-Out & Grain Integrity:** Verifying that non-aggregated projection columns appear in `GROUP BY` clauses and detecting cartesian products.
- **Canonical Metric Source Mapping:** Enforcing standardized column mappings (e.g., mapping revenue to `order_items.price`).

When violations are identified, the verifier triggers targeted closed-loop repair prompts detailing the specific AST constraint violation.

# 4 Experimental Methodology

## 4.1 Data Warehouse & Benchmark Corpus

We evaluate our system on an enterprise-style relational schema constructed from the Brazilian E-Commerce Public Dataset (Olist [6]):

- **9 Relational Tables:** `customers`, `orders`, `order_items`, `order_payments`, `order_reviews`, `products`, `sellers`, `geolocation`, and `product_category_name_translation`.
- **Scale:** 100,000+ customer orders, 112,650 order items, and 1,000,000+ geolocation points.
- **Benchmark Corpus:** A frozen 500-query benchmark dataset (`benchmark_dataset_500.json`) stratified across 8 business domains and 3 difficulty tiers (Easy: 114, Medium: 276, Hard: 110).

## 4.2 Evaluation Metrics & Methodological Definitions

- **Result Equivalence Rate under the Study Comparator (95% Wilson CI):** Evaluates whether executed SQL results match ground-truth result sets under row-order invariance (comparing row multisets via item Counters) with numerical tolerance ($\epsilon = 0.01$) and string whitespace normalization. Positional column semantics are preserved. We emphasize that result equivalence is an empirical evaluation criterion under the study comparator, not a generic classification accuracy or formal mathematical proof of semantic correctness.
- **Exact Match Rate:** Strict character-for-character equality of raw SQL output rows and column headers.
- **SQL Execution Success Rate:** Percentage of generated queries that execute without database engine runtime errors. Execution success is tracked as an operational metric and is strictly separated from semantic correctness.
- **Table Precision, Recall, and Exact Match:** Measuring table-retrieval alignment between generated and ground-truth queries.

# 5 Main 500-Query Results

## 5.1 Headline Performance

Table 1 reports the headline results across all 500 benchmark queries.

The system achieves a **73.4% Result Equivalence Rate under the study comparator** (69.3%–77.2%, 95% Wilson Score CI; Clopper-Pearson Exact CI: [69.30%, 77.22%]), **31.0% Exact Match**, and **100.0% SQL Execution Success** with zero provider timeouts, HTTP 429 exceptions, or unhandled errors. Mean query execution latency is 64.04s ($p50$: 56.47s, $p95$: 121.92s).

Table 1: Headline Benchmark Performance (500 Queries)

| Metric | Phase 10 Live Run (95% CI) |
|---|---|
| Result Equivalence Rate | **73.4%** [69.3%, 77.2%] |
| Exact Match Rate | 31.0% [27.0%, 35.3%] |
| SQL Execution Success | **100.0%** [99.1%, 100.0%] |
| Table Exact Accuracy | 82.6% |
| Table Precision | 93.1% |
| Table Recall | 95.3% |
| Mean Latency | 64.04s (p95: 121.92s) |

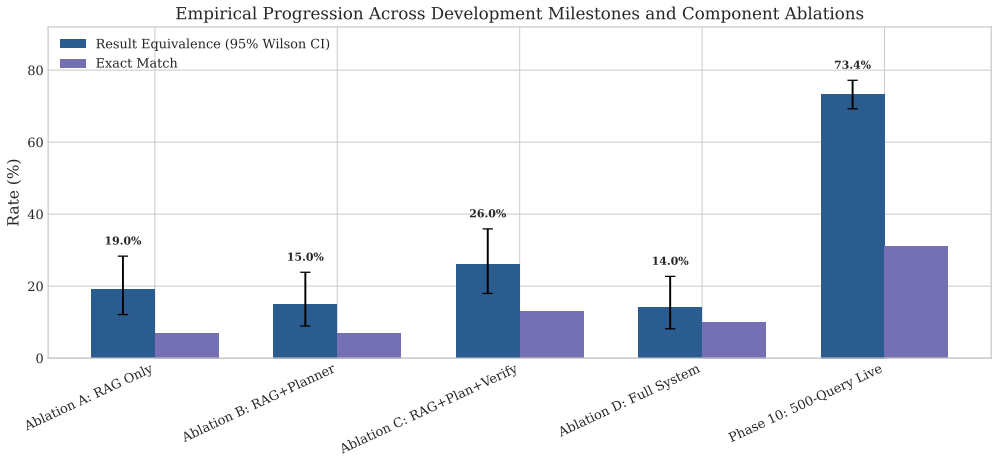


Figure 2: Empirical progression across development milestones and component configurations with 95% Wilson score confidence intervals.

Figure 2 depicts the longitudinal progression across development milestones, which we report as historical engineering context rather than clean causal evidence, as multiple system components evolved across phases. Figure 3 shows the accuracy–latency configuration frontier.

### 5.2 Domain and Difficulty Stratification

Performance varies across business domains and difficulty tiers (Table 2 and Figure 4):

- **Domain Differences:** *Orders & Transactions* (90.8%) and *Sellers & Fulfillment* (91.7%) achieve high result equivalence due to direct primary-foreign key links. Conversely, *Reviews & Satisfaction* (46.7%) and *Logistics & Operations* (56.7%) present lower equivalence due to complex multi-table bridge joins and date-arithmetic nuances.
- **Difficulty Tiers:** Easy queries achieve 88.60% equivalence (101/114), Medium queries achieve 76.81% (212/276), and Hard queries drop to 44.55% (49/110).
- **Query Type Breakdown:** Single-value queries achieve 90.30% (242/268), time-series queries achieve 87.76% (43/49), ranked-list queries achieve 48.57% (68/140), and aggregated tables achieve 32.56% (14/43).

## 6 Controlled Component Ablation

To measure the marginal impact of individual pipeline stages, we evaluated four controlled configurations across 100 identical benchmark query instances (Table 3):

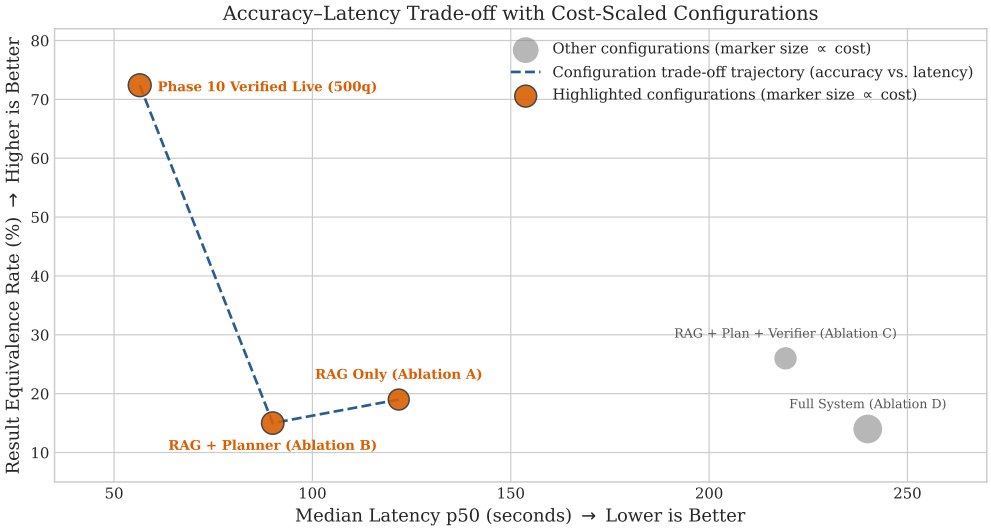


Figure 3: Accuracy–Latency trade-off across evaluated configurations (marker size proportional to estimated token cost).

Table 2: Performance Breakdown Across 8 Business Domains (Phase 10, N=500)

| Domain | N | Result Equivalence (95% CI) | SQL Exec | Table Prec | Table Rec | Mean Latency |
|---|---|---|---|---|---|---|
| Customers & Geography | 65 | 80.0% [67.9%, 88.5%] | 100.0% | 99.5% | 97.4% | 56.7s |
| Logistics & Operations | 60 | 56.7% [43.3%, 69.2%] | 100.0% | 94.2% | 100.0% | 61.7s |
| Orders & Transactions | 65 | 90.8% [80.3%, 96.2%] | 100.0% | 99.2% | 100.0% | 68.4s |
| Payments & Installments | 60 | 71.7% [58.4%, 82.2%] | 100.0% | 80.8% | 81.7% | 54.9s |
| Products & Categories | 65 | 76.9% [64.5%, 86.1%] | 100.0% | 94.1% | 100.0% | 63.2s |
| Revenue & Sales | 65 | 70.8% [58.0%, 81.1%] | 100.0% | 97.7% | 92.3% | 74.8s |
| Reviews & Satisfaction | 60 | 46.7% [33.9%, 59.9%] | 100.0% | 88.9% | 90.6% | 66.6s |
| Sellers & Fulfillment | 60 | 91.7% [80.9%, 96.9%] | 100.0% | 88.6% | 100.0% | 65.4s |

- **Config A (RAG Only):** Direct prompt-based generation from retrieved schema; verifier disabled.
- **Config B (RAG + Planner):** LLM DAG planner with plan validation; verifier disabled.
- **Config C (RAG + Planner + Verifier):** LLM DAG planner with AST structural verifier and repair active.
- **Config D (Full System):** Multi-stage pipeline including post-execution evaluator agent.

**Ablation Findings.** Config B (adding an unverified planner) causes execution success to collapse from 99.0% to 34.0% and result equivalence to drop from 19.0% to 15.0%, because unconstrained multi-step DAG planning introduces complex structural and alias errors. Activating the AST Structural Verifier (Config C) substantially restores reliability: execution success rises from 34.0% to 65.0% (+31.0%), and result equivalence improves from 15.0% to 26.0% (+11.0%).

In a matched paired McNemar test on the 100 identical queries, Config C demonstrates a statistically significant improvement over Config B (exact binomial $p = 0.0192 < 0.05$, Odds Ratio $= 3.75$, with 15 queries solved only by Config C vs. 4 solved only by Config B). We note that this ablation isolates the marginal stabilizing effect of structural verification over planning, but does not claim that the ablation alone proves the entire live system architecture causally superior.

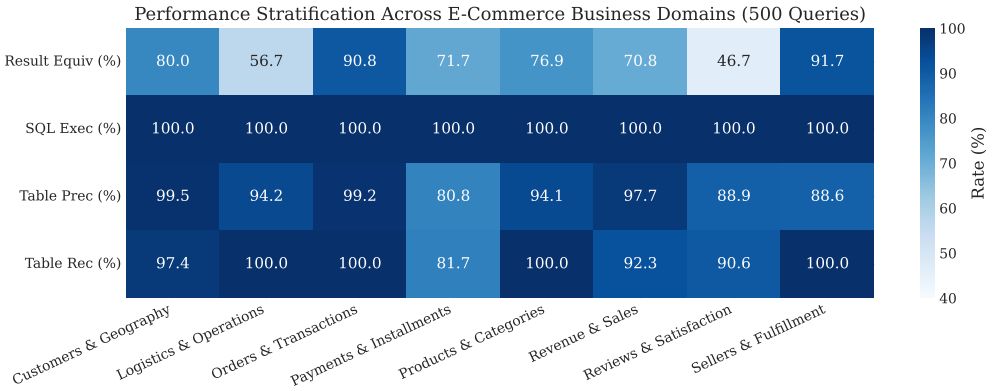


Figure 4: Performance Stratification Across Business Domains and Difficulty Tiers (500 Queries).

Table 3: Longitudinal Comparison Across Development Phases

| Phase / System Variant | N | Result Equivalence (95% CI) | Exact Match | SQL Exec | Table Rec | Latency (s) |
|---|---|---|---|---|---|---|
| Ablation A: RAG Only | 100 | 19.0% [12.1, 28.3] | 7.0% | 99.0% | 0.0% | 153.0 (p95: 309.1) |
| Ablation B: RAG+Planner | 100 | 15.0% [8.9, 23.9] | 7.0% | 34.0% | 0.0% | 85.8 (p95: 93.1) |
| Ablation C: RAG+Plan+Verify | 100 | 26.0% [18.0, 35.9] | 13.0% | 65.0% | 0.0% | 209.4 (p95: 240.0) |
| Ablation D: Full System | 100 | 14.0% [8.1, 22.7] | 10.0% | 45.0% | 0.0% | 232.6 (p95: 271.3) |
| Phase 10: 500-Query Live | 500 | 73.4% [69.3, 77.2] | 31.0% | 100.0% | 95.3% | 64.0 (p95: 121.9) |

# 7 Granular Audit of the 101 Repair Cases

To evaluate the true dynamics of automated self-repair, we conducted an exhaustive audit of all 101 repair events triggered during the 500-query benchmark run (Table 4 and Figure 5).

**Key Empirical Findings on Automated Repair:**

1. **Execution Success Is Not Semantic Recovery:** While 96.0% (97/101) of post-repair queries executed cleanly on SQLite, only 52.5% (53/101) produced correct result sets. Describing compiler execution validity as repair recovery is methodologically invalid.

2. **The False-Positive Regression Hazard:** Out of 101 triggered repair events, 71 queries were already semantically correct before repair was invoked. For 22 of these queries (21.8% of all repair events), the automated repair agent degraded a correct query into an incorrect one (e.g., by injecting erroneous date filters or altering valid join aliases).

3. **Genuine Recovery Rate:** Automated repair genuinely rescued only 4 previously broken queries (`q_291`, `q_346`, `q_442`, `q_476`), while preserving 49 already-correct queries and failing to resolve 26 incorrect queries.

This finding represents a central empirical takeaway: unrestricted rule-based repair risks inducing substantial false-positive regressions, indicating that conservative, uncertainty-gated verification is preferable to unconstrained repair loops.

# 8 AST Failure Taxonomy

We analyzed all 133 non-equivalent queries from the 500-query evaluation using AST diffing against ground-truth SQL (Figure 6):

1. **Schema Missing Join Path (37 queries, 27.8% of errors / 7.4% overall):** Omission of required intermediate bridging tables (e.g., joining `order_reviews` to `customers` without traversing `orders`).

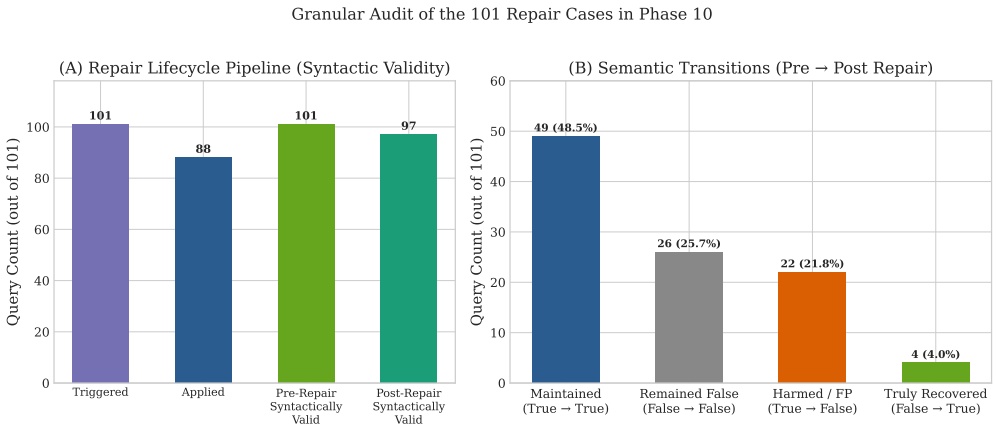


Figure 5: Granular Audit of the 101 Repair Cases: Syntactic Validity Pipeline (Panel A) and Pre → Post Semantic Transitions (Panel B: 49 maintained, 26 remained incorrect, 22 harmed false positives, 4 truly recovered).

Table 4: Granular Lifecycle & Semantic Transition Audit of 101 Repair Cases

| **Repair Stage / Transition** | **Count** | **Share (%)** |
|---|---|---|
| Repair Triggered by Verifier | 101 | 100.0% |
| Repair Applied | 88 | 87.1% |
| Post-Repair Syntactically Valid | 97 | 96.0% |
| **Semantic Transitions (Pre → Post Repair)** | | |
| Maintained Correct (True → True) | 49 | 48.5% |
| Remained Incorrect (False → False) | 26 | 25.7% |
| Harmed / False Positive (True → False) | 22 | 21.8% |
| Truly Recovered (False → True) | **4** | **4.0%** |

2. **Semantic Filter Omission or Error (33 queries, 24.8% of errors / 6.6% overall):** Missing domain-specific predicates (e.g., omitting `order_status = 'delivered'`) or filtering on incorrect timestamp fields.

3. **Semantic Aggregation Mismatch (32 queries, 24.1% of errors / 6.4% overall):** Using incorrect aggregation functions (`AVG` instead of `SUM`) or inconsistent currency rounding.

4. **Schema Hallucinated Table (15 queries, 11.3% of errors / 3.0% overall):** Synthesizing nonexistent subqueries or table aliases absent from the schema catalog.

5. **Grain / GROUP BY Mismatch (14 queries, 10.5% of errors / 2.8% overall):** Grouping by string aliases rather than canonical date expressions, causing grain collision.

6. **Semantic Ranking Order Mismatch (2 queries, 1.5% of errors / 0.4% overall):** Inverted `ASC`/`DESC` ordering or missing `ORDER BY` in ranking queries.

## 9 Controlled Synthetic Perturbation Robustness

To examine sensitivity to controlled lexical and semantic shifts, we evaluated a deterministic 50-query synthetic perturbation suite ($N = 50$, 10 queries per vector across 8 domains; Table 5 and Figure 7). We emphasize that this suite tests specific local perturbation robustness rather than broad out-of-distribution or cross-domain generalization.

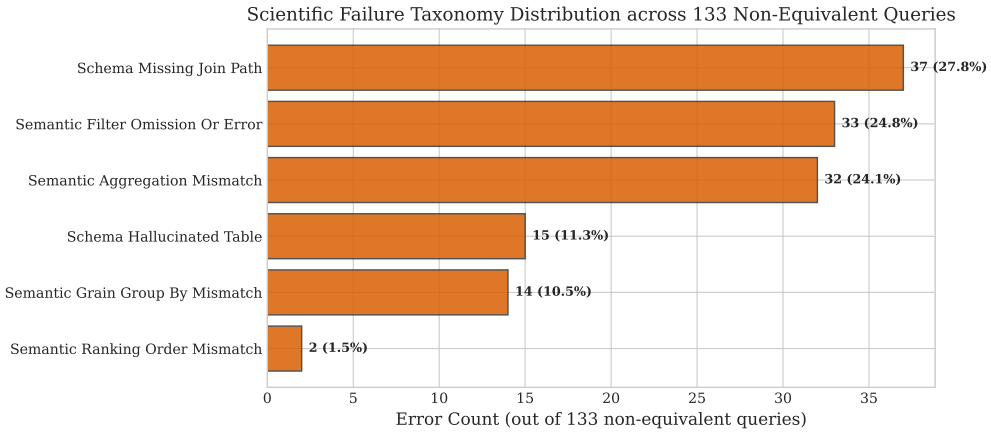


Figure 6: Scientific Failure Taxonomy Distribution across 133 Non-Equivalent Queries.

Table 5: Controlled Synthetic Perturbation Robustness Under 5 Vectors

| Perturbation Type | N | Clean Acc | Perturbed Acc | ΔAcc | Retention Rate |
|---|---|---|---|---|---|
| Paraphrase | 10 | 50.0% | 40.0% | -10.0% | 80.0% |
| Ranking Variant | 10 | 70.0% | 70.0% | -0.0% | 100.0% |
| Synonym | 10 | 80.0% | 80.0% | -0.0% | 100.0% |
| Temporal Variant | 10 | 70.0% | 60.0% | -10.0% | 85.7% |
| Typo | 10 | 70.0% | 40.0% | -30.0% | 57.1% |

**Robustness Observations.** The pipeline demonstrates high retention under query paraphrasing (80.0% retention), ranking phrasing variants (100.0% retention), and synonym substitution (100.0% retention). Temporal boundary shifts exhibit moderate degradation (85.7% retention). However, character-level typographical noise causes severe degradation (57.1% retention, 30.0% absolute drop), exposing the vulnerability of exact token-matching in the schema retriever.

## 10 Limitations and Threats to Validity

We explicitly document the scientific and operational limitations of this study:

1. **Single Data Warehouse Schema:** All evaluations were conducted on the Olist e-commerce relational schema (9 tables, 100k orders). While architecturally representative of relational star/snowflake schemas, our results do not claim zero-shot transfer to financial, healthcare, or graph database schemas without domain-specific schema RAG tuning.

2. **Custom Frozen Benchmark:** The 500-query benchmark is a custom frozen corpus. Although stratified across domains and difficulty tiers, evaluations on external public benchmarks (e.g., Spider, BIRD) remain necessary for cross-benchmark comparison.

3. **Inference Latency Overhead:** Multi-stage retrieval, planning, validation, and verification incur a mean latency of 64.04s ($p95$: 121.92s), making this architecture suited for asynchronous analytics rather than sub-second interactive search.

4. **Hard Query Degradation:** Result equivalence drops to 44.55% on hard queries and 32.56% on aggregated table queries, reflecting persistent limitations in multi-step CTE nesting and window functions.

5. **Typographical Sensitivity:** The schema retriever degrades significantly under typographical noise (57.1% retention), indicating the necessity of fuzzy edit-distance indexing.

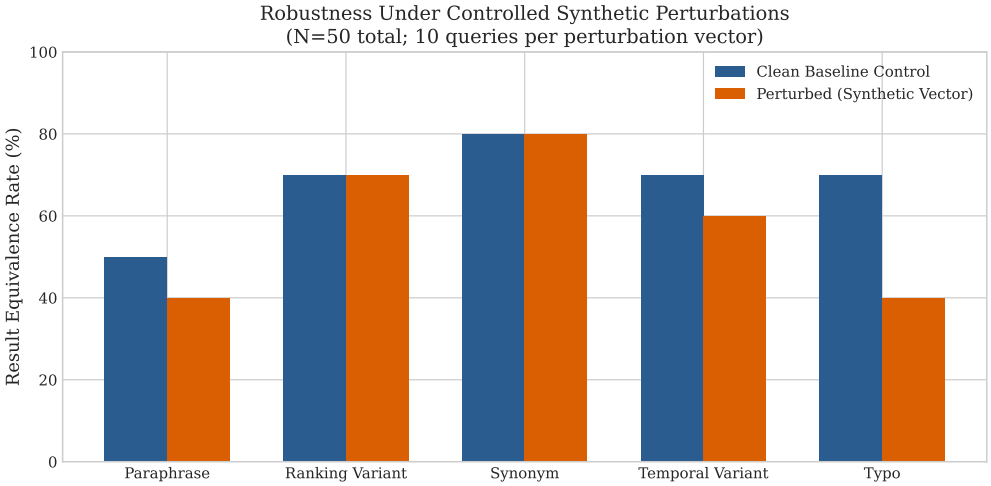


Figure 7: Robustness under controlled synthetic perturbations ($N = 50$ total; 10 queries per perturbation vector).

6. **False-Positive Repair Regressions:** Heuristic repair rules induced regressions in 22 out of 101 triggered cases, underscoring the need for uncertainty-aware gating before applying automated repairs.

7. **Empirical Comparator Scope:** The row-multiset comparator is an empirical evaluation metric with numerical tolerance under the defined comparator, not a formal mathematical proof of semantic correctness.

8. **Ablation Sample Sizes:** Component ablations ($N = 100$) and synthetic perturbation tests ($N = 50$) use smaller sample sizes than the main 500-query benchmark.

## 11 Reproducibility and Open Artifacts

All code, benchmark definitions, evaluation scripts, and manuscript sources are open-source:

- Complete reproduction instructions in `REPRODUCIBILITY.md`.
- Deterministic database builder in `data/build_database.py`.
- Cryptographic artifact manifest with SHA-256 hashes in `ARTIFACT_MANIFEST.json`.
- Open-source MIT code license and Olist CC BY-NC-SA 4.0 data attribution in `LICENSE`.

## 12 Conclusion

In this study, we investigated the architectural mechanisms governing the reliability of LLM-generated analytical SQL over a public relational e-commerce data warehouse. On an audited 500-query benchmark, our multi-stage pipeline achieved a **73.4% Result Equivalence Rate under the study comparator** and **100.0% SQL Execution Success**. Our controlled component ablation demonstrated that AST-based structural verification significantly stabilizes unverified query planners ($p = 0.0192$). However, an exhaustive audit of 101 repair cases revealed that automated self-repair is a double-edged mechanism, producing 22 harmful false-positive regressions against only 4 genuine recoveries. These findings demonstrate that while reliability-oriented grounding, planning, and structural verification significantly improve analytical SQL synthesis, conservative, uncertainty-aware verification is critical to prevent automated repair regressions.

## References


[1] Akari Asai, Min Sewon, Zexuan Zhong, and Danqi Chen. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023.

[2] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[4] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls (bird). *Advances in Neural Information Processing Systems*, 36:25341–25356, 2023.

[5] Toby Mao and Contributors. Sqlglot: An uncompromising sql parser and transpiler. `https://github.com/tobymao/sqlglot`, 2023.

[6] Olist and Kaggle. Brazilian e-commerce public dataset by olist. `https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce`, 2018. Licensed under CC BY-NC-SA 4.0.

[7] Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems*, 36:36338–36350, 2023.

[8] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

[9] Bing Wang, Changyu Ren, Jian Zheng, Yong Zhang, Zhen Liang, and Heyan Lin. Mac-sql: A multi-agent collaborative framework for text-to-sql. *arXiv preprint arXiv:2312.11242*, 2024.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[11] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018.